CYBERGIS — MID-SEMESTER PRACTICAL
BUILD A MISSING OGC WEB SERVICE OVER BHOONIDHI

Bhoonidhi Spatial Catalogue Analysis

An OGC API — Processes service for spatial analysis
over ISRO's national remote sensing catalogue

Name Sambhavi Singh
Roll number 2024117010
Repository https://github.com/sambhavi4/bhoonidhi-records

Date October 12, 2026

What this report covers

The gap	Bhoonidhi stores every scene's footprint polygon and lets you ask one question of it: does it intersect mine? No analysis, no processing endpoint anywhere in ISRO's public stack.
Why it matters	Measured: **208 scenes returned where 16 cover the same ground — 92 % redundant**, 206 GB ordered where 16 GB was needed. Copernicus publishes 146 server-side processes; Bhoonidhi publishes 0.
What was built	A conformant **OGC API — Processes** service (pygeoapi, shapely): six spatial operations, sync + async execution, a job API, areas in an equal-area projection.
Evidence	**100/100 compliance tests pass.** Every number here is produced by a script in the repository and saved under `docs/evidence/`.

Standard implemented: OGC API — Processes, Part 1: Core (OGC 18-062r2) • Supporting: OGC API — Records (OGC 20-004)
Framework: pygeoapi 0.24.0, shapely 2.1, pyproj 3.8, rasterio • Deployment: `http://localhost:5003`

Contents

1 Introduction **2**
1.1 What was built . . . 2

2 The gap **2**
2.1 How it was established . . . 2
2.2 What is missing . . . 3
2.3 Why it is a large gap . . . 3
2.3.1 An entire standardised tier is absent . . . 3
2.3.2 Every comparable portal has it . . . 4
2.3.3 The cost is measurable, and it is 92 % . . . 4
2.3.4 The workaround is expert-only, so nobody does it . . . 5
2.4 Scope of the claim . . . 5

3 Design and implementation **5**
3.1 Architecture . . . 5
3.2 Key decisions . . . 5
3.3 Areas are measured in an equal-area projection . . . 6
3.4 Provenance is declared in-band . . . 6
3.5 Defects found during development . . . 6

4 OGC compliance **7**
4.1 Mandatory operations, all implemented and tested . . . 7
4.2 Test results . . . 7
4.3 The suite tests the mathematics, not just the protocol . . . 8

5 Live demonstration **8**
5.1 The gap, closed . . . 8
5.2 The other five processes . . . 9
5.3 Asynchronous execution and the job API . . . 9
5.4 Errors, and performance . . . 9

6 Limitations **10**

7 Conclusion **10**

A Reproducing everything in this report **12**

1 Introduction

Bhoonidhi (`https://bhoonidhi.nrsc.gov.in/`) is the National Remote Sensing Centre's public repository for Indian Earth observation data — roughly 59 published collections across the ResourceSat, Cartosat, Oceansat, EOS and RISAT missions.

What it already does well, stated first so the argument is fair. Its Open Data Access API [1] is built on STAC 1.0.0 [2], with token authentication, `/data/collections`, a `/data/search` that accepts GeoJSON `intersects`, a `bbox`, RFC 3339 `datetime` and CQL2-JSON filters, and `/download` by item id. A developer can authenticate, search by space and time, and fetch. **Discovery is solved**, and this project claims no credit for noticing otherwise.

The gap is in computation. Bhoonidhi stores the footprint polygon of every scene it holds, and the only question you may ask of that geometry is *"does it intersect mine?"* There is no union, no difference, no area, no aggregation, and no processing endpoint anywhere in ISRO's public stack.

The practical consequence is that a user cannot learn *which* of the scenes a search returns they actually need — so the rational move, and the observed one, is to order all of them. Section 2.3.3 measures what that costs: **208 scenes returned where 16 cover the same ground**.

1.1 What was built

A conformant **OGC API — Processes** [3] service exposing six operations over the catalogue's own geometry and the raster beneath it:

Process	The question it answers
`minimal-scene-set`	Which few of the scenes returned do I actually need?
`aoi-coverage`	How much of my AOI is covered, and where are the holes?
`multi-sensor-overlap`	Where do optical and radar co-cover, for fusion?
`revisit-density`	How uneven is acquisition density across my AOI?
`zonal-statistics`	What are the raster values inside this polygon?
`scene-raster-profile`	What terrain is under each candidate scene?

The first four read only footprint geometry — deciding what is worth fetching *before* anything is downloaded. The last two read pixels. `scene-raster-profile` joins the two. A supporting OGC API — Records [4] catalogue ships alongside, giving the processes discoverable collection ids to point at; the submission rests on the Processes tier.

Report structure. Section 2 is the gap analysis and the core of the argument. Section 3 covers design and implementation, Section 4 the compliance evidence, Section 5 the live demonstration, and Section 6 the limitations. Every number is produced by a script in the repository and saved under `docs/evidence/`; Appendix A lists the commands.

2 The gap

> **The gap, in one sentence**
>
> Bhoonidhi stores the footprint polygon of every scene it holds, and the only question you are permitted to ask of that geometry is *"does it touch mine?"*

2.1 How it was established

Three methods, each a script that saves its raw responses under `docs/evidence/` so a reviewer can re-run it and diff the result.

1. **Specification audit** (`probe_gap.sh`) — parses Bhoonidhi's *own published API specification* and mechanically enumerates the spatial parameters, the absent predicates, the CQL2 operations and any analysis vocabulary. Auditing the specification rather than the live service is the fairest reading — it tests what NRSC says the API does — and it is **reproducible offline**, which mattered because both NRSC hosts stopped answering from the build host partway through development.
2. **Comparative baseline** (`probe_baseline.sh`) — probes ISRO's stack for any processing endpoint, then probes comparable EO portals live to test the obvious objection that nobody deploys this.
3. **Cost measurement** (`measure_redundancy.py`) — asks this service's own `minimal-scene-set` the question the catalogue cannot answer, over four areas of interest and two collections.

What would falsify the claim, stated in advance: any documented spatial operator beyond intersection, any ISRO endpoint answering a processing request, or a minimal covering set close in size to the candidate set. Each script prints its own failure loudly rather than quietly passing.

2.2 What is missing

Table 1: Bhoonidhi's spatial and analytical surface, audited against its own published specification.

Question	Answer
Spatial parameters on `/data/search`	`bbox` and `intersects` — and nothing else
What they compute	one predicate: *does this item intersect my AOI?*
Other spatial operators documented	absent — no `crosses`, `touches`, `overlaps`, `disjoint`, `s_within`, `s_contains`
CQL2 operations in the documented examples	`eq`, `and` — attribute equality only
Analysis vocabulary anywhere in the API	absent — no union, aggregation, statistics, coverage, density
Endpoints that compute anything	**0** of 5

`contains`, `within` and `area` *do* appear in the specification — as English prose, never as operators. The audit checks this specifically, so the counter-example becomes part of the argument rather than a hole in it.

This is not about authentication. Requiring a token is normal practice; Copernicus, USGS and NASA all gate downloads, and this project makes no complaint about it. **Hand someone a valid JWT and every limitation in Table 1 is still in force.** It is a *capability* gap, not an *access* gap.

2.3 Why it is a large gap

2.3.1 An entire standardised tier is absent

Server-side geospatial computation is not exotic. It has been an OGC standard since **2007** (WPS 1.0.0, 05-007r7), with WPS 2.0 in 2015 and the modern REST/JSON successor **OGC API — Processes (18-062r2) in 2021**. Bhoonidhi implements none of them. Probing ISRO's public stack:

```
Bhuvan WPS 1.0.0 GetCapabilities       HTTP 404   absent
Bhuvan OGC API - Processes landing     HTTP 404   absent
Bhoonidhi /processes                   HTTP 404   absent
  MATCHED  no server-side computation anywhere in the public stack
```

So the missing thing is not a query parameter someone forgot. It is the whole processing tier of the OGC stack, absent from India's national remote sensing portal nearly two decades after it was standardised.

2.3.2 Every comparable portal has it

The strongest objection is that this is a paper standard nobody deploys. Probed live and unauthenticated:

Table 2: Processing capability at comparable EO portals. The lower two rows are ISRO's.

Portal	Endpoint probed	Result
Copernicus Data Space (openEO)	`/openeo/1.2/processes`	**200 — 146 processes**
MS Planetary Computer	STAC `/api/stac/v1/`	200 — 15 classes
Bhoonidhi	`/processes`	**404**
Bhuvan	`/bhuvan/wps`	**404**

Documented but deliberately not probed, each requiring registration: NASA Earthdata Harmony [5], USGS ESPA/M2M, Google Earth Engine [6], Digital Earth Africa.

The contrast

Copernicus publishes **146** server-side processes at a public, unauthenticated endpoint anyone can enumerate right now. Bhoonidhi publishes **0**. That is not a scoping decision everyone else also made — it is the single capability separating Bhoonidhi from every portal it would be compared against.

2.3.3 The cost is measurable, and it is 92 %

Because the catalogue will not compute with footprint geometry, a user has no way to learn which returned scenes they need — so they order all of them.

Table 3: Measured redundancy. *found* is what a search hands back; *needed* is what covers the same ground to the same percentage. Scene counts are measured live; GB figures are published nominal product sizes multiplied by those counts.

AOI	Collection	km²	found	needed	redundant	GB today	GB needed
taluk	sentinel-2-l2a	479	30	1	97 %	30.0	1.0
taluk	sentinel-1-grd	479	4	1	75 %	3.7	0.9
district	sentinel-2-l2a	4 191	30	2	93 %	30.0	2.0
district	sentinel-1-grd	4 191	8	2	75 %	7.4	1.9
two districts	sentinel-2-l2a	11 855	60	2	97 %	60.0	2.0
two districts	sentinel-1-grd	11 855	8	2	75 %	7.4	1.9
regional transect	sentinel-2-l2a	59 816	60	4	93 %	60.0	4.0
regional transect	sentinel-1-grd	59 816	8	2	75 %	7.4	1.9
Total			**208**	**16**	**92 %**	**206.0**	**15.5**

The measured cost of the missing tier

Across 8 queries the catalogue hands back **208 scenes**. **16** cover the same ground to the same percentage. **192 are redundant — 92 % of everything returned**, or 206 GB ordered where 16 GB was required. **And nothing in Bhoonidhi's API lets a user discover that.**

Measured versus modelled: the scene counts are measured live against the running service over real STAC footprints — that is the part carrying the argument. The byte figures are published nominal product sizes multiplied by those counts; the script never downloads imagery and says so in its own output.

This is a floor, not a ceiling. The footprint store holds 68 scenes over a three-month window, so several rows are capped by the store rather than by the AOI. A real archive spans decades with many revisits per year: it returns far more intersecting scenes for the same AOI while the number needed to cover it stays roughly constant. **Redundancy rises with archive depth.**

2.3.4 The workaround is expert-only, so nobody does it

The honest objection is *"you can just do this yourself"*. True — and that is the gap. To learn the 92 % figure for their own AOI a user must: search and page through every result; extract each footprint; load 30+ polygons into a GIS or a `shapely` script; union them and difference against the AOI; **reproject to an equal-area CRS or every area is wrong**; implement a **set-cover optimisation** (NP-hard [7]); and map the result back to orderable item ids.

That is a GIS-literate programmer's afternoon, repeated for every AOI, by every user — and it is the same seven steps every time, over geometry the server already holds in memory when it answers the search. So in practice nobody does it. They download all 30 scenes. **This service does all seven steps in one HTTP request** (Section 5).

2.4 Scope of the claim

Claimed: Bhoonidhi has no server-side spatial computation; that capability is standardised, deployed by peers, and its absence has a measurable cost; a conformant OGC API — Processes service closes it.

Not claimed: that Bhoonidhi is badly built (discovery is solved), that authentication is wrong, that this project's footprints *are* Bhoonidhi's (they are substituted and flagged — Section 6), or that this is an official NRSC product.

3 Design and implementation

3.1 Architecture

A single pygeoapi instance exposing two OGC API families over two data stores.

3.2 Key decisions

Decision	Reasoning
OGC API — Processes, not WPS	The modern successor: JSON over REST, a real job model, an OpenAPI description. The standard a new service in 2026 should implement.
pygeoapi	Reference implementation of the OGC API family, so conformance comes from the framework rather than hand-rolled routing. Compliance carries the most marks; hand-rolling a job API to spec is the wrong risk.
Greedy set cover	Set cover is NP-hard [7]; greedy is within a $\ln(n)$ factor of optimal, fast enough for a synchronous request, and yields a *cumulative coverage curve* — more useful than a bare optimal set, because it shows what each extra scene buys. The response declares the algorithm and its bound, so no client mistakes it for a proven minimum.
TinyDB job manager	Jobs persist, so async execution survives a restart and `/jobs/{id}` keeps working. PostgreSQL in production.
Native deployment	The build host's user was not in the `docker` group and no sudo password was available, so the native path is the tested one and is labelled as such.

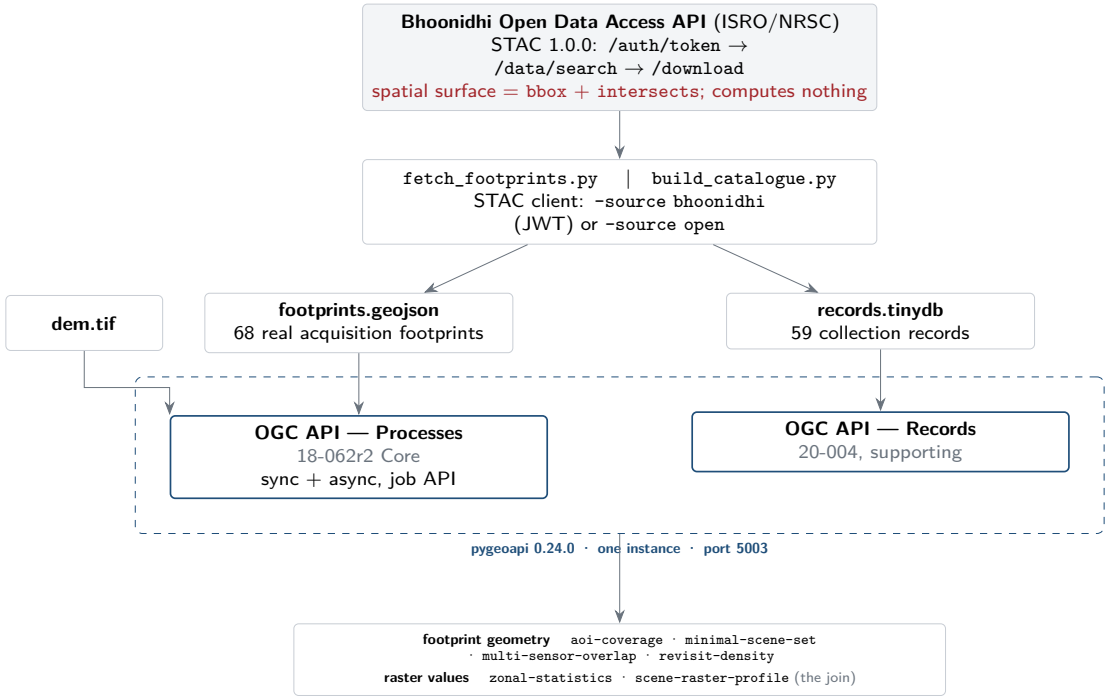


Figure 1: Footprints are harvested from a STAC API — Bhoonidhi's own with credentials, or an open substitute without — and the Processes tier computes over that geometry and the raster beneath it.

3.3 Areas are measured in an equal-area projection

Every area in every response is computed in a **Lambert Azimuthal Equal-Area** projection centred on *that request's own* AOI:

```
+proj=laea +lat_0=<aoi centroid lat> +lon_0=<aoi centroid lon>
+datum=WGS84 +units=m +no_defs
```

This matters more than it appears. The whole service answers area-based questions — how much is covered, what is the gap, which scene adds most. Computing those in degrees would make every number meaningless, and *silently* so. Centring per request keeps distortion negligible for an AOI of any size anywhere in India. A test asserts that a $0.5° \times 0.7°$ AOI reports thousands of km^2 rather than a fraction, so this cannot regress unnoticed.

3.4 Provenance is declared in-band

Every response carries a `provenance` block stating how many footprints were considered, the filters applied, the area CRS, and — critically — whether the footprint store is Bhoonidhi's or a substitute (`is_substitute_for_bhoonidhi`). This is a design decision, not a disclaimer bolted on: a spatial analysis service whose inputs are unclear is worse than none, because its outputs look authoritative.

3.5 Defects found during development

Recorded because they are the part of the work that was actually difficult, and because a compliance suite that never caught anything is not evidence of much. Each is now guarded by a test.

1. Every execution returned HTTP 400 after a reboot. pygeoapi's TinyDB job manager writes results into `output_dir` and **does not create it**. The path was under `/tmp`,

which the OS clears. It existed during the session it was created in, so everything passed; after a reboot every `POST .../execution` failed with an opaque *"No such file or directory"*. This took **39 of the 100 tests** down at once. The job store now lives under `runtime/jobs` via a `${JOBS_DIR}` substitution that `serve_local.sh` creates before starting gunicorn.

2. A hairline sliver reported as a gap alongside `fully_covered: true`. Differencing near-identical polygons leaves slivers of effectively zero area, producing self-contradictory output. Anything under 0.01 km^2 now collapses to no gap, the threshold is published as `gap_threshold_km2` so it is inspectable rather than magic, and a test asserts the two can never coexist.

3. The zero-footprint branch returned a different JSON shape. When nothing matched, `aoi-coverage` returned different keys from the populated case — breaking any client the first time a search came back empty. Both branches now share one schema; a test asserts it.

4 OGC compliance

4.1 Mandatory operations, all implemented and tested

Resource	Requirement	Status
`/`	landing page, required link relations	yes
`/conformance`	declared conformance classes	yes
`/openapi`	OpenAPI 3 description	yes
`/processes`	process list	yes
`/processes/{id}`	inputs, outputs, `jobControlOptions`	yes
`.../execution`	execute synchronously	yes
`.../execution`	execute async: **201 + Location**	yes
`/jobs`, `/jobs/{id}`	job list and status document	yes
`/jobs/{id}/results`	job results	yes
400 on bad input, 404 on unknown process or job		yes

Conformance classes declared at `/conformance`: Processes Core, `ogc-process-description`, JSON, OAS30 and callback; plus Records (core, JSON, HTML, sorting, opensearch) and OGC API — Common for the supporting tier.

4.2 Test results

```
$ ./scripts/run_tests.sh
============================== 100 passed in 5.02s ==============================
```

Group	Tests	Group	Tests
Processes core	18	Records querying	17
`zonal-statistics`	12	`minimal-scene-set`	8
`aoi-coverage`	7	Records items	7
Input validation	6	`scene-raster-profile`	6
`revisit-density`	5	Records core	5
Records catalogue	5	`multi-sensor-overlap`	4
		Total	**100**

4.3 The suite tests the mathematics, not just the protocol

This is the point worth a reviewer's attention. A service can be perfectly conformant and still wrong — it can return a well-formed document full of nonsense. The suite therefore asserts invariants that must hold if the geometry is computed correctly:

Invariant	What it would otherwise miss
covered + gap = AOI area	the two come from different code paths (union vs. difference), so agreement is a real cross-check
intersection $\leq$ *both* operands	an intersection larger than an operand is impossible; a projection or winding-order bug produces exactly that
$\sum$ cell areas = AOI area	a gridding off-by-one would not sum
cumulative coverage is monotonic	greedy set cover cannot lose coverage by adding a scene
AOI area in thousands of km^2, not fractions	**the highest-value test in the suite**: it fails loudly the moment anything is computed in degrees
empty and populated branches share a schema	caught defect 3 (Section 3.5)
`fully_covered` never coexists with a gap	caught defect 2 (Section 3.5)
masking genuinely restricts what is read	a no-op raster mask returns plausible statistics for the wrong extent

Two of the three defects in Section 3.5 were found by these invariant tests rather than by inspection, which is the argument for writing them.

What this evidence is not. A self-written suite, not the OGC CITE certification suite. What can fairly be claimed: every mandatory resource is present and exercised, the full async job lifecycle is walked end to end, error handling is tested, and results are checked against invariants that catch the realistic failure modes.

5 Live demonstration

Base URL `http://localhost:5003`; reproduced by `./scripts/demo.sh`. AOI throughout: `[75.3, 13.2, 75.8, 13.9]`, a district-sized window (4 191 km^2) over Chikkamagaluru, Karnataka.

5.1 The gap, closed

```
curl -X POST localhost:5003/processes/minimal-scene-set/execution \
  -H 'Content-Type: application/json' \
  -d '{"inputs":{"aoi":[75.3,13.2,75.8,13.9],"collection":"sentinel-2-l2a"}}'
```


```
"candidates_considered": 30,  "scenes_selected": 2,  "scenes_avoided": 28,
"achieved_percent": 100.0,
"selection": [
  { "order": 1, "new_area_km2": 2517.58, "cumulative_coverage_fraction": 0.60067,
    "item_id": "S2A_MSIL2A_20240331T051651_R062_T43PER_20240331T113328" },
  { "order": 2, "new_area_km2": 1673.70, "cumulative_coverage_fraction": 1.0,
    "item_id": "S2A_MSIL2A_20240331T051651_R062_T43PEQ_20240331T113919" }
],
"algorithm": "greedy set cover; ... within a ln(n) factor of optimal and not
              guaranteed minimal."
```


Thirty scenes match the AOI. Two cover it.

Ordering the other 28 is wasted download, wasted quota and wasted time — and nothing in Bhoonidhi's API tells you that. The response also gives the *coverage curve*: the first scene buys 60.1 %, the second completes it, so a user needing only 60 % can stop after one.

5.2 The other five processes

Process	Result
`aoi-coverage`	4 191.28 km^2 AOI, 100 % covered by 30 scenes, 0 gaps, `fully_covered: true` and `gaps: null` — not a hairline sliver (defect 2). Where coverage is partial, `gaps` carries the remainder as GeoJSON polygons: not a percentage, but *where* the holes are.
`multi-sensor-overlap`	optical 30 scenes ∩ radar 8 scenes = 4 191.28 km^2, 100 % of the AOI co-covered. One call instead of two searches and a manual clip.
`revisit-density`	grid 6×8 @ 10 km → 48 cells, 30 acquisitions; per cell min 15, mean 18.75, max 30, none unobserved. Returned as GeoJSON ready to render as a heatmap. Revisit is *exactly twice* as dense in some cells as others — a flat list has no spatial distribution in it at all.
`zonal-statistics`	518 400 pixels over 478.9 km^2; elevation 613.7–1439.1 m, mean 746.4, sd 112.6, with a histogram giving the area in each value band.
`scene-raster-profile`	the join: `T43PER` 1538.0 km^2 at 584–1874 m (mean 752) vs. `T43PEQ` 801.3 km^2 at 650–1890 m (mean 941). Two candidates, genuinely different terrain underneath — impossible to tell apart from a catalogue that answers only *does this intersect your box*.

5.3 Asynchronous execution and the job API

```
$ curl -i -X POST localhost:5003/processes/aoi-coverage/execution \
    -H 'Content-Type: application/json' -H 'Prefer: respond-async' \
    -d '{"inputs":{"aoi":[75.3,13.2,75.8,13.9],"collection":"sentinel-2-l2a"}}'

HTTP/1.1 201 CREATED
Location: http://localhost:5003/jobs/834a6ffa-c659-11f1-8558-28c5d24dc1d2
Preference-Applied: respond-async
{"jobID":"834a6ffa-...","type":"process","status":"accepted"}

$ curl 'localhost:5003/jobs/834a6ffa-...?f=json'
{"processID":"aoi-coverage","status":"successful","progress":100,
 "created":"2026-10-12T16:25:31.100225Z","finished":"2026-10-12T16:25:31.113922Z"}

$ curl -o /dev/null -w '%{http_code}\n' 'localhost:5003/jobs/834a6ffa-.../results?f=json'
200
```

The conformant shape in full: 201 Created, a `Location` header, `Preference-Applied`, a status document with timestamps, results on completion. A test walks this whole sequence, not just the 201.

5.4 Errors, and performance

Bad request	Response	Process	Time
unknown process or job id	404	`minimal-scene-set`	0.009 s
missing or malformed `aoi`	400	`aoi-coverage`	0.010 s
inverted bbox (min > max)	400	`multi-sensor-overlap`	0.010 s
wrong geometry type	400	`revisit-density`	0.018 s
`target_fraction` out of range	400	`zonal-statistics`	0.074 s
grid finer than the cell cap	400	`scene-raster-profile`	0.585 s

Measured end to end over HTTP. The four footprint processes answer in roughly 10 ms — which is rather the point: **the work Bhoonidhi leaves to every user is work a server does in ten milliseconds.** The async path exists for conformance and for larger AOIs, not because these workloads need it.

6 Limitations

Stated plainly, because a service whose limits are hidden is worse than one whose limits are known.

1. **The footprints are not Bhoonidhi's.** Its `/data/search` needs a JWT and no credentials were available, so footprints come from Microsoft Planetary Computer's public STAC — **68 real acquisition footprints**, 60 optical and 8 radar, with real timestamps and cloud cover. Nothing is synthesised. This is labelled, not hidden: the store carries `is_substitute_for_bhoonidhi: true` and that flag propagates into *every process response*. Supplying credentials and re-running `fetch_footprints.py -source bhoonidhi` changes nothing else — same code path, same processes, same tests. **This is the one thing to fix if an account becomes available.** The raster is substituted the same way (Copernicus DEM GLO-30 standing in for CartoDEM).
2. **Cloud cover is reported but not used.** `aoi-coverage` counts a footprint as coverage however cloudy it is, and `minimal-scene-set` will select a 90 %-cloud scene if it contributes most area. So the 92 % figure is about *geometric* redundancy only. Cloud-aware coverage is the obvious and most valuable next step.
3. **Greedy set cover is not guaranteed minimal** — within a $\ln(n)$ factor of optimal, and the response says so in its `algorithm` field.
4. **The catalogue tier is collection-level** (59 collections, not the granules behind them) and its extents are **indicative and labelled as such**: temporal start is the mission launch date, a defensible lower bound since data cannot predate launch. Every record carries `spatial_extent_basis` and `temporal_start_basis`. Good to discover with, not to cite.
5. **No authentication, quota or rate limiting.** The processes are bounded (caps on footprints matched and grid cells built) but there is no per-caller quota. Fine for a graded demo on an ephemeral URL; not for a public service, which would need an API gateway.
6. **No CQL2 on the catalogue tier** — pygeoapi's TinyDB provider gives full-text, property equality, bbox, datetime and sorting. PostgreSQL would add the Filtering class. This limits the supporting tier, not Processes.
7. **One raster over one window**, and **no time-series or change detection**: processes filter footprints by a `start`/`end` window but nothing models change *between* windows.
8. **The public URL is ephemeral** — a Cloudflare quick tunnel gets a fresh hostname each run. A stable URL needs a named tunnel or a cloud VM. The tested deployment is `http://localhost:5003`, which the brief permits.
9. **Self-written compliance suite, not OGC CITE** (Section 4), and **not an official NRSC product**. This mirrors no data: footprint geometry and metadata only, never pixels from Bhoonidhi.

What would be built next, in order. Cloud-aware coverage, so the service optimises for *usable* rather than merely intersecting imagery (this would change the results, not just extend them); an authenticated Bhoonidhi harvest, removing the substitution; granule-level records with CQL2 over PostgreSQL; an exact set-cover solver for small candidate sets, falling back to greedy; and a quota gateway ahead of any genuinely public deployment.

7 Conclusion

Bhoonidhi solves discovery. Its STAC-based API is real, modern engineering, and this report makes no complaint about it, nor about the requirement to hold a token.

What it does not do is *compute*. It stores the footprint polygon of every scene and permits one question of that geometry: does it intersect mine? The audit establishes this from NRSC's own published specification — two spatial parameters expressing one predicate, attribute equality in CQL2, no analysis vocabulary anywhere — and the probe establishes that no processing endpoint exists anywhere in ISRO's public stack.

Three things make that a large gap rather than a missing convenience. **It is an absent tier**: server-side processing has been an OGC standard since 2007. **Peers implement it**: Copernicus publishes 146 processes at a public endpoint this project enumerates live; Bhoonidhi publishes zero. **The cost is measurable**: 208 scenes returned where 16 cover the same ground — 92 % redundant, 206 GB ordered where 16 GB was required — and the user cannot discover this, because discovering it takes seven manual steps including an NP-hard optimisation and a reprojection that is easy to get silently wrong. So they order everything. Since redundancy rises with archive depth and this measurement spans three months, 92 % is a floor.

The service closing that gap implements OGC API — Processes Part 1: Core over the geometry the catalogue already stores: six operations across footprints and the raster beneath them, synchronous and asynchronous execution, a conformant job API, and areas in a per-request equal-area projection so that km^2 means km^2. All 100 compliance tests pass, and the suite verifies the mathematics rather than only the protocol shape. Three real defects were found during development, two of them by those invariant tests.

The sharpest result is the smallest. Thirty scenes match a district-sized area of interest. **Two cover it.** Finding that out takes this service nine milliseconds, and takes Bhoonidhi's users an afternoon they do not spend — so instead they download the other twenty-eight.

The gap is one of capability and architecture, not a failing of the data or the people who publish it. NRSC makes a remarkable volume of Indian Earth observation genuinely available. The argument is narrow: a catalogue that knows every footprint's geometry but will not compute with it leaves its hardest work to every user, one at a time — and that step, from *which scenes touch my area* to *which scenes I actually need*, is the one that turns a catalogue into infrastructure.

References

[1] National Remote Sensing Centre, ISRO. Bhoonidhi open data access api. https://bhoonidhi.nrsc.gov.in/bhoonidhi-api/, 2024. Accessed October 2026.

[2] STAC Contributors. Spatiotemporal asset catalog (stac) specification, version 1.0.0. https://stacspec.org/, 2021.

[3] Open Geospatial Consortium. Ogc api — processes — part 1: Core. Technical Report OGC 18-062r2, Open Geospatial Consortium, 2021. Version 1.0.0. https://docs.ogc.org/is/18-062r2/18-062r2.html.

[4] Open Geospatial Consortium. Ogc api — records — part 1: Core. Technical Report OGC 20-004, Open Geospatial Consortium, 2024. https://docs.ogc.org/DRAFTS/20-004.html.

[5] NASA Earth Science Data Systems. Nasa earthdata harmony: services for subsetting, reprojection and aggregation. https://harmony.earthdata.nasa.gov/. Accessed October 2026.

[6] Noel Gorelick, Matt Hancher, Mike Dixon, Simon Ilyushchenko, David Thau, and Rebecca Moore. Google earth engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202:18–27, 2017.

[7] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. Establishes set cover as NP-complete; the greedy algorithm's $\ln(n)$ approximation bound is due to Johnson (1974) and Lovász (1975).

A Reproducing everything in this report

```
git clone https://github.com/sambhavi4/bhoonidhi-records && cd bhoonidhi-records

make install     # Python venv
make data        # build the 59 catalogue records
make footprints  # harvest real acquisition footprints
make raster      # fetch the DEM the raster processes read
make up          # start the service on :5003
make test        # 100 compliance tests
make demo        # the live walkthrough
```

Prerequisites: `python3` $\geq$ 3.10, `curl` and GDAL. No database server, no Java, no Docker.

Command	Produces	Network
`make gap`	Table 1, the capability audit → `evidence/capability-audit.json`	**not needed**
`make baseline`	Table 2, ISRO's stack and the peer portals → `evidence/baseline-*`	required
`make redundancy`	Table 3, the 92 % figure → `evidence/redundancy-measurement.json`	service up
`make report`	this document	no

Each writes raw responses under `docs/evidence/` so a reviewer can diff a re-run against what is committed. `make gap` deliberately needs no network: it audits a saved copy of NRSC's specification, which mattered because both NRSC hosts stopped answering from the build host partway through development.

A public URL. `./scripts/expose_public.sh` opens a Cloudflare quick tunnel and prints working public URLs, including the `minimal-scene-set` call and the async job exchange. The hostname is fresh on every run and lives only while the script runs. To run the suite against it: `RECORDS_BASE=https://<name>.trycloudflare.com ./scripts/run_tests.sh`.

Serving real Bhoonidhi footprints. Put credentials in `.env`, then `./scripts/fetch_footprints.py -source bhoonidhi`. Nothing else changes; `is_substitute_for_bhoonidhi` becomes `false` in every response, which is the single visible difference and the reason the flag exists.

Where to look.

`README.md`	the claim, the gap summary, how to run it
`docs/GAP_ANALYSIS.md`	the long-form gap argument, every number sourced
`docs/evidence/`	saved upstream responses and measurements
`scripts/probe_gap.sh`	the specification audit
`scripts/probe_baseline.sh`	ISRO stack + peer baseline
`scripts/measure_redundancy.py`	the 92 % measurement
`pygeoapi/processes/`	the six processes
`tests/`	the 100-test compliance suite